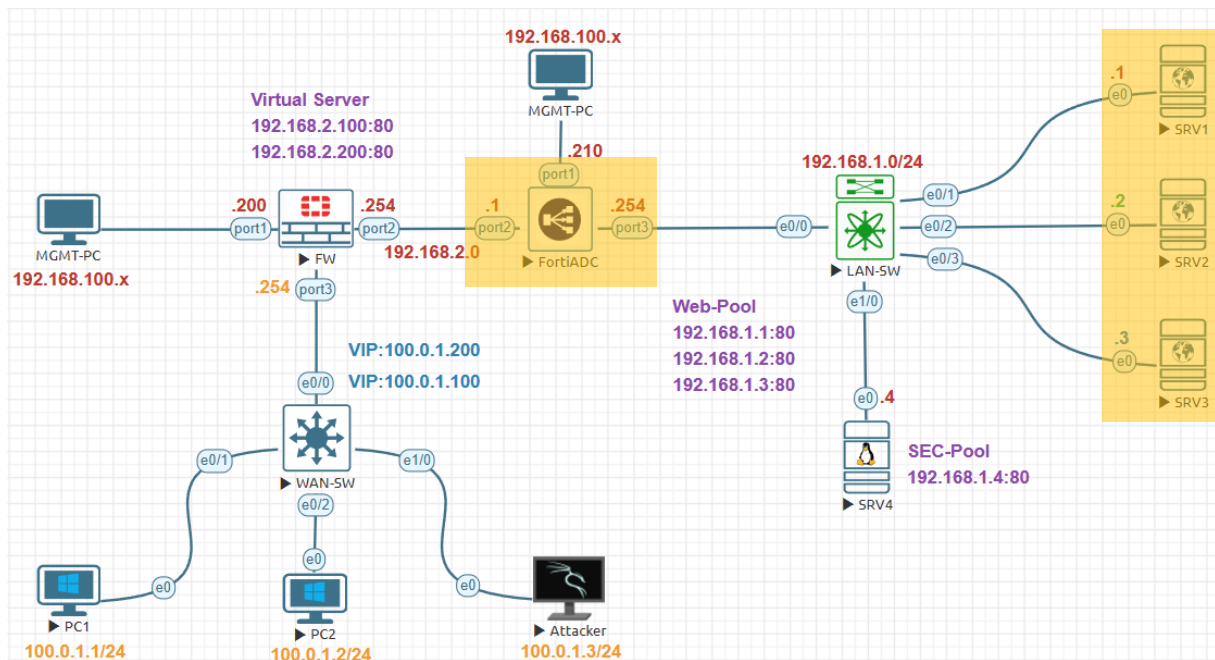


TCP Health Check Lab:



Management Subnet	192.168.100.0/24
FortiGate Management IP	192.168.100.200
FortiADC Management IP	192.168.100.210
Internal Servers Subnet	192.168.1.0/24
FortiADC External Subnet	192.168.2.0/24
FortiGate Firewall External	100.0.1.0/24
FortiADC Internal IP Address	192.168.1.254
FortiADC External IP Address	192.168.2.1
FortiGate Firewall Internal IP	192.168.2.254
FortiGate Firewall External IP	100.0.1.254
SRV1 IP Address	192.168.1.1
SRV2 IP Address	192.168.1.2
SRV3 IP Address	192.168.1.3
SRV3 IP Address	192.168.1.4
Virtual Sever-1	192.168.2.100
Virtual Sever-2	192.168.2.200
FortiGate VIP-1	100.0.1.100
FortiGate VIP-2	100.0.1.200
External PC1 IP Address	100.0.1.1
External PC2 IP Address	100.0.1.2
External PC3 IP Address	100.0.1.3

Let's create our own custom Health Check Navigate to Shared **Resources>Health Check>Health Check** click on **Create New**

FortiADC FortiADC

Dashboard > Health Check Health Check Monitor Health Check Script

Security Fabric > Delete Create New Add Filter

FortiView >

System >

Shared Resources >

Health Check

Name	Type
LB_HLTHCK_ICMP	ICMP
LB_HLTHCK_HTTP	HTTP
LB_HLTHCK_HTTPS	HTTPS
LB_HLTHCK_TCP_ECHO	TCP Echo

Showing 1 to 4 of 4 entries 0 rows selected Show 25 entries

Network >

Type **Name** in this case **TCP-HealthCheck** and choose Type **TCP** click **Save**.

Health Check

Dashboard >

Security Fabric >

FortiView >

System >

Shared Resources >

Health Check

Schedule Group

Address

Service

Network >

Server Load Balance >

Link Load Balance >

Global Load Balance >

Web Application Firewall >

Network Security >

DoS Protection >

User Authentication >

Log & Report >

Name TCP-HealthCheck

Type TCP

Specifics

Port 80

Range: 0-65535

General

Destination Address Type IPv4 IPv6

Destination Address 0.0.0.0

Example: 192.0.2.1

Up Retry 1

Default: 1 Range: 1-10 retries

Down Retry 3

Default: 3 Range: 1-10 retries

Interval 5

Default: 5 Range: 1-3600 seconds

Timeout 3

Default: 3 Range: 1-3600 seconds

Save

Next navigate to **Server Load Balance > Real Server Pool > Real Server Pool** tab double click on **WEB-Pool** to open the setting.

Real Server Pool Real Server Server SSL

Delete + Create New + Add Filter

Name	Address Type	Health Check
WEB-Pool	IPv4	Enable
SEC-Pool	IPv4	Disable

Showing 1 to 2 of 2 entries 0 rows selected Show 25 entries

Server Load Balance

- Virtual Server
- Application Resources
- Application Optimization
- Real Server Pool**

Toggle to enable the **Health Check** on right side Double-click to choose **TCP-HealthCheck** previously created click **Save** to apply the Health Check it will inherited automatically.

Real Server Pool

Name: WEB-Pool

Address Type: IPv4 IPv6

Type: Static Dynamic

Health Check: ☒

Health Check Relationship: AND OR

Health Check List: TCP-HealthCheck

Available Items: LB_HLTHCK_ICMP, LB_HLTHCK_HTTP, LB_HLTHCK_HTTPS, LB_HLTHCK_TCP_ECHO, ICMP-HealthCheck

Double-click to deselect. Drag to reorder. Double-click to select.

Direct Route Mode: ☐

Real Server SSL Profile: NONE

Member

ID	Name	Address	Health Check
1	SRV1	192.168.1.1	inherited
2	SRV2	192.168.1.2	inherited
3	SRV3	192.168.1.3	inherited

Delete + Create New + Add Filter

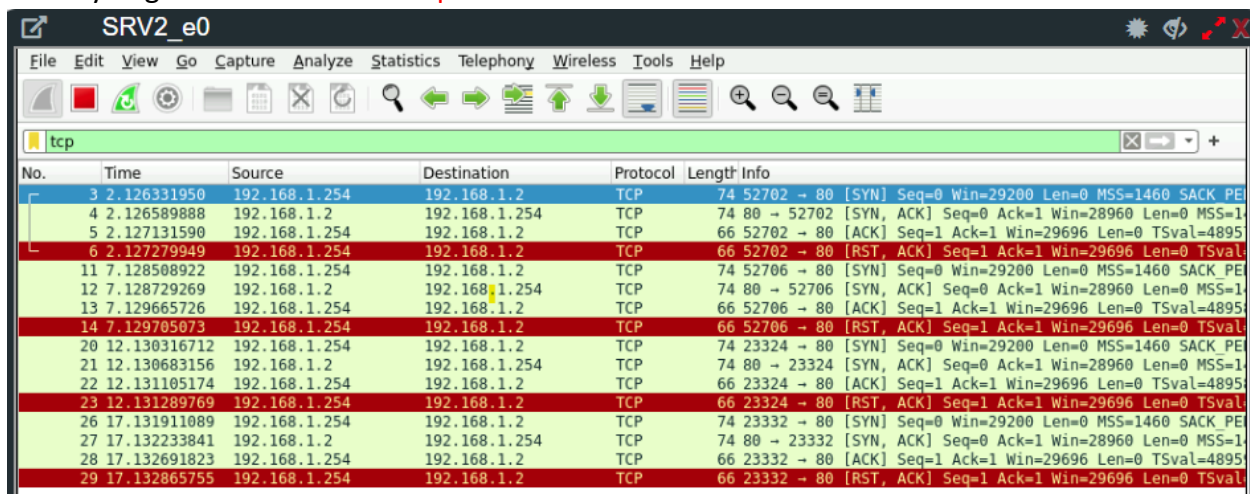
Save Cancel

Verification:

Go to SRV1 login with username & password root/root type **tcpdump tcp**.

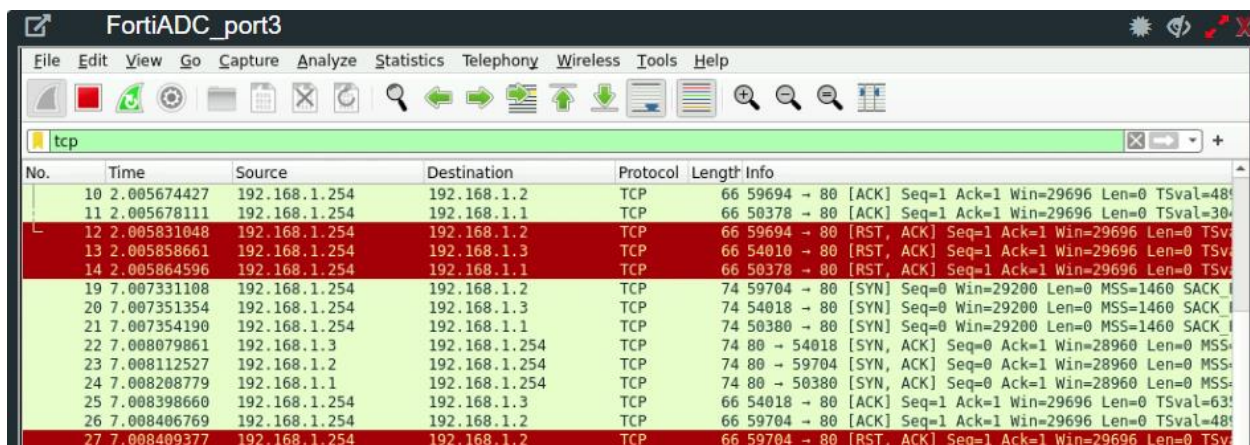
```
SRV1 x
root@SRV1:~# tcpdump tcp
tcpdump: verbose output suppressed, use -v or -vv for full
listening on ens3, link-type EN10MB (Ethernet), capture size
13:24:44.878562 IP 192.168.1.254.59034 > 192.168.1.1.http:
05007 ecr 0,nop,wscale 9], length 0
13:24:44.878653 IP 192.168.1.1.http > 192.168.1.254.59034:
ckOK,Ts val 309071 ecr 3048305007,nop,wscale 7], length 0
13:24:44.879506 IP 192.168.1.254.59034 > 192.168.1.1.http:
length 0
13:24:44.879523 IP 192.168.1.254.59034 > 192.168.1.1.http:
90711. length 0
```

Alternatively, you can use the built-in Wireshark capture tool in EVE-NG to monitor TCP traffic directly. Right click one **SRV2 > Capture** choose interface **e0**



The screenshot shows the Wireshark interface for SRV2_e0. The packet list table contains the following data:

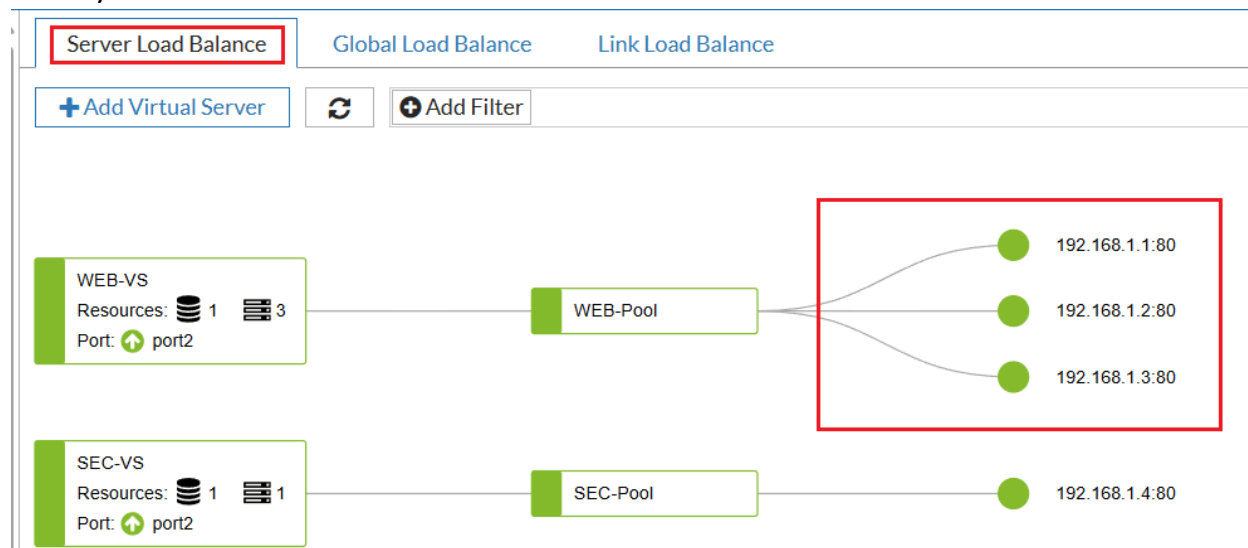
No.	Time	Source	Destination	Protocol	Length	Info
3	2.126331950	192.168.1.254	192.168.1.2	TCP	74	52702 → 80 [SYN] Seq=0 Win=29200 Len=0 MSS=1460 SACK PE
4	2.126589888	192.168.1.2	192.168.1.254	TCP	74	80 → 52702 [SYN, ACK] Seq=0 Ack=1 Win=28960 Len=0 MSS=1
5	2.127131590	192.168.1.254	192.168.1.2	TCP	66	52702 → 80 [ACK] Seq=1 Ack=1 Win=29696 Len=0 TSval=4895
6	2.127279949	192.168.1.254	192.168.1.2	TCP	66	52702 → 80 [RST, ACK] Seq=1 Ack=1 Win=29696 Len=0 TSval
11	7.128508922	192.168.1.254	192.168.1.2	TCP	74	52706 → 80 [SYN] Seq=0 Win=29200 Len=0 MSS=1460 SACK PE
12	7.128729269	192.168.1.2	192.168.1.254	TCP	74	80 → 52706 [SYN, ACK] Seq=0 Ack=1 Win=28960 Len=0 MSS=1
13	7.129665726	192.168.1.254	192.168.1.2	TCP	66	52706 → 80 [ACK] Seq=1 Ack=1 Win=29696 Len=0 TSval=4895
14	7.129705073	192.168.1.254	192.168.1.2	TCP	66	52706 → 80 [RST, ACK] Seq=1 Ack=1 Win=29696 Len=0 TSval
20	12.130316712	192.168.1.254	192.168.1.2	TCP	74	23324 → 80 [SYN] Seq=0 Win=29200 Len=0 MSS=1460 SACK PE
21	12.130683156	192.168.1.2	192.168.1.254	TCP	74	80 → 23324 [SYN, ACK] Seq=0 Ack=1 Win=28960 Len=0 MSS=1
22	12.131105174	192.168.1.254	192.168.1.2	TCP	66	23324 → 80 [ACK] Seq=1 Ack=1 Win=29696 Len=0 TSval=4895
23	12.131289769	192.168.1.254	192.168.1.2	TCP	66	23324 → 80 [RST, ACK] Seq=1 Ack=1 Win=29696 Len=0 TSval
26	17.131911089	192.168.1.254	192.168.1.2	TCP	74	23332 → 80 [SYN] Seq=0 Win=29200 Len=0 MSS=1460 SACK PE
27	17.132233841	192.168.1.2	192.168.1.254	TCP	74	80 → 23332 [SYN, ACK] Seq=0 Ack=1 Win=28960 Len=0 MSS=1
28	17.132691823	192.168.1.254	192.168.1.2	TCP	66	23332 → 80 [ACK] Seq=1 Ack=1 Win=29696 Len=0 TSval=4895
29	17.132865755	192.168.1.254	192.168.1.2	TCP	66	23332 → 80 [RST, ACK] Seq=1 Ack=1 Win=29696 Len=0 TSval



The screenshot shows the Wireshark interface for FortiADC_port3. The packet list table contains the following data:

No.	Time	Source	Destination	Protocol	Length	Info
10	2.005674427	192.168.1.254	192.168.1.2	TCP	66	59694 → 80 [ACK] Seq=1 Ack=1 Win=29696 Len=0 TSval=489
11	2.005678111	192.168.1.254	192.168.1.1	TCP	66	50378 → 80 [ACK] Seq=1 Ack=1 Win=29696 Len=0 TSval=30
12	2.005831048	192.168.1.254	192.168.1.2	TCP	66	59694 → 80 [RST, ACK] Seq=1 Ack=1 Win=29696 Len=0 TSv
13	2.005858661	192.168.1.254	192.168.1.3	TCP	66	54010 → 80 [RST, ACK] Seq=1 Ack=1 Win=29696 Len=0 TSv
14	2.005864596	192.168.1.254	192.168.1.1	TCP	66	50378 → 80 [RST, ACK] Seq=1 Ack=1 Win=29696 Len=0 TSv
19	7.007331108	192.168.1.254	192.168.1.2	TCP	74	59704 → 80 [SYN] Seq=0 Win=29200 Len=0 MSS=1460 SACK I
20	7.007351354	192.168.1.254	192.168.1.3	TCP	74	54018 → 80 [SYN] Seq=0 Win=29200 Len=0 MSS=1460 SACK I
21	7.007354190	192.168.1.254	192.168.1.1	TCP	74	50380 → 80 [SYN] Seq=0 Win=29200 Len=0 MSS=1460 SACK I
22	7.008079861	192.168.1.3	192.168.1.254	TCP	74	80 → 54018 [SYN, ACK] Seq=0 Ack=1 Win=28960 Len=0 MSS
23	7.008112527	192.168.1.2	192.168.1.254	TCP	74	80 → 59704 [SYN, ACK] Seq=0 Ack=1 Win=28960 Len=0 MSS
24	7.008208779	192.168.1.1	192.168.1.254	TCP	74	80 → 50380 [SYN, ACK] Seq=0 Ack=1 Win=28960 Len=0 MSS
25	7.008398660	192.168.1.254	192.168.1.3	TCP	66	54018 → 80 [ACK] Seq=1 Ack=1 Win=29696 Len=0 TSval=63
26	7.008406769	192.168.1.254	192.168.1.2	TCP	66	59704 → 80 [ACK] Seq=1 Ack=1 Win=29696 Len=0 TSval=489
27	7.008409377	192.168.1.254	192.168.1.2	TCP	66	59704 → 80 [RST, ACK] Seq=1 Ack=1 Win=29696 Len=0 TSv

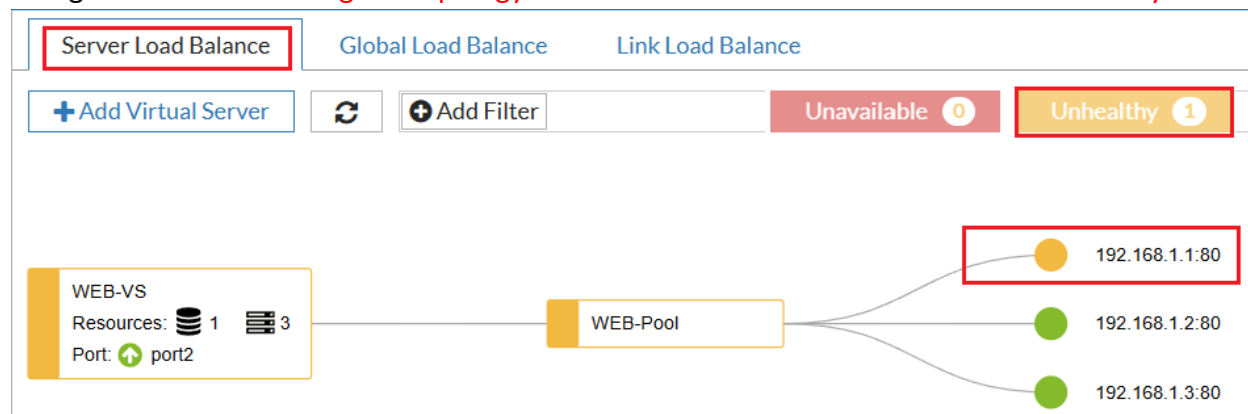
Navigate to **FortiView>Logical Topology>Server Load Balance** all the servers are up green and healthy.



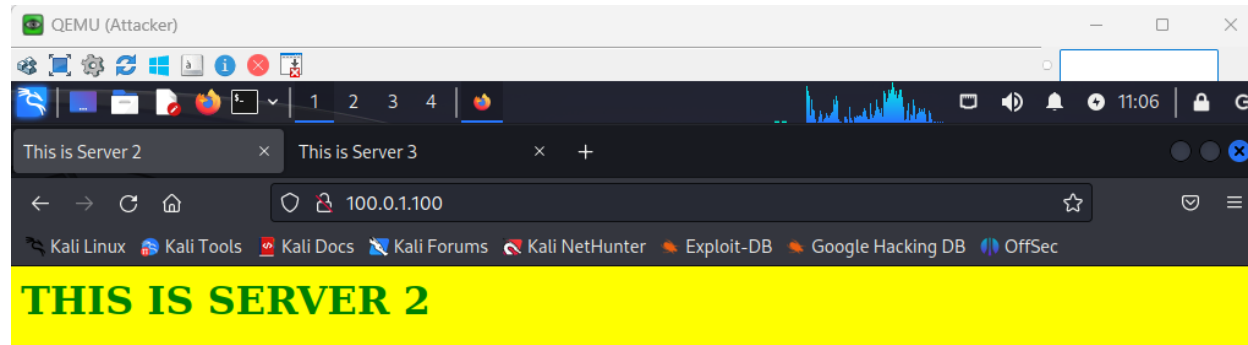
To test the TCP-Based Health check let's stop one of the Server in this case **SRV1**.

root@SRV1:~# shutdown -h now

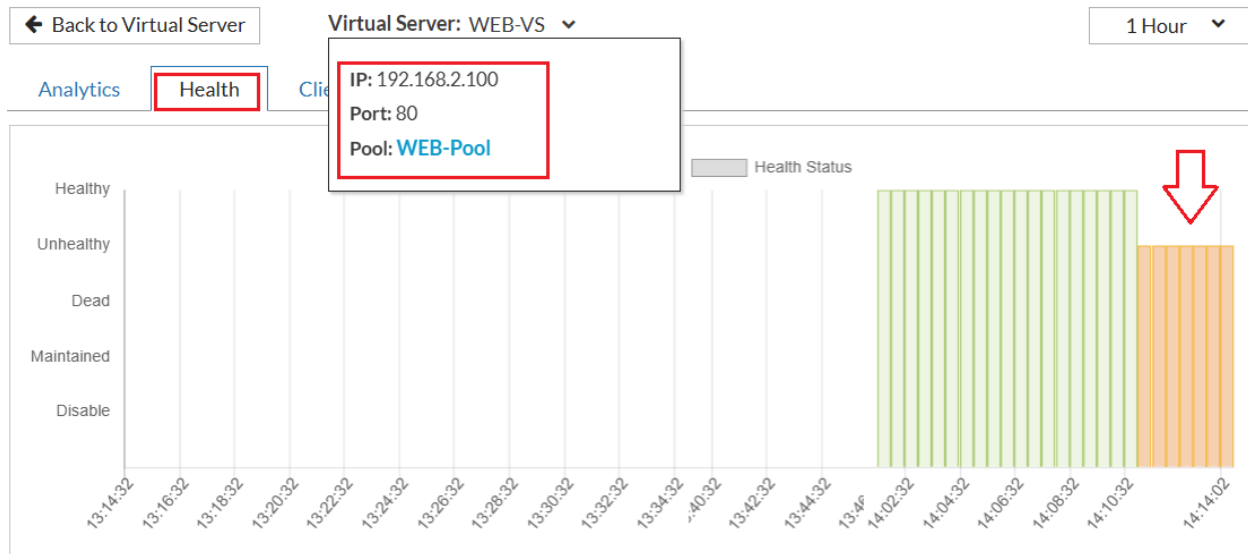
Navigate to **FortiView>Logical Topology>Server Load Balance** this time **SRV1** is unhealthy



Based on the health FortiADC is not sending traffic to **Server 1** only to SERVER 2 and SERVER 3



Navigate to **FortiView>Virtual Server** click on the name of Virtual Server in this case **WEB-VS** after that click on **Health** tab to check Health Status the organ color show is unhealthy



Let's start **SERV1**, but this time, stop only the **Web Services** running on it. This approach allows us to simulate a scenario where the server itself is operational but the application or web service is unavailable.

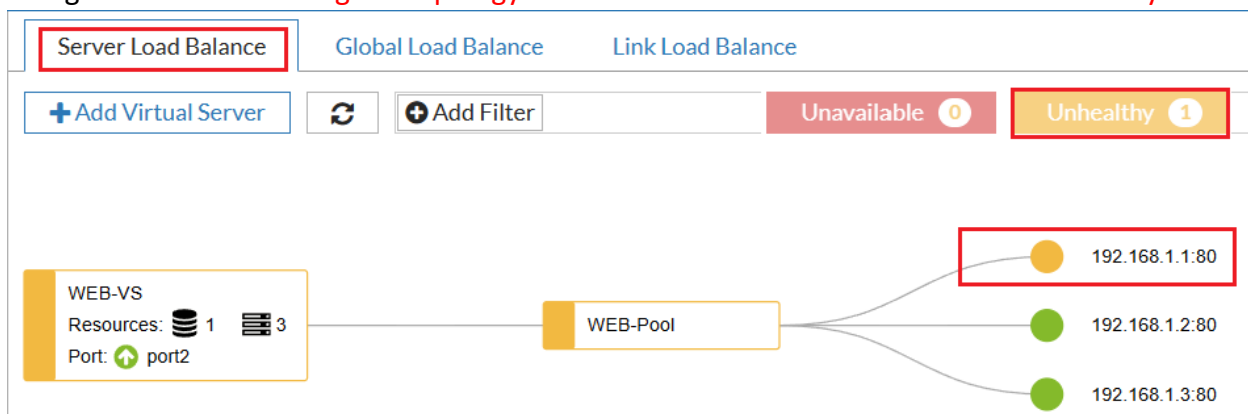
root@SRV1:~# service apache2 stop

```

root@SRV1:~# service apache2 stop
root@SRV1:~#

```

Navigate to **FortiView>Logical Topology>Server Load Balance** this time **SERV1** is unhealthy

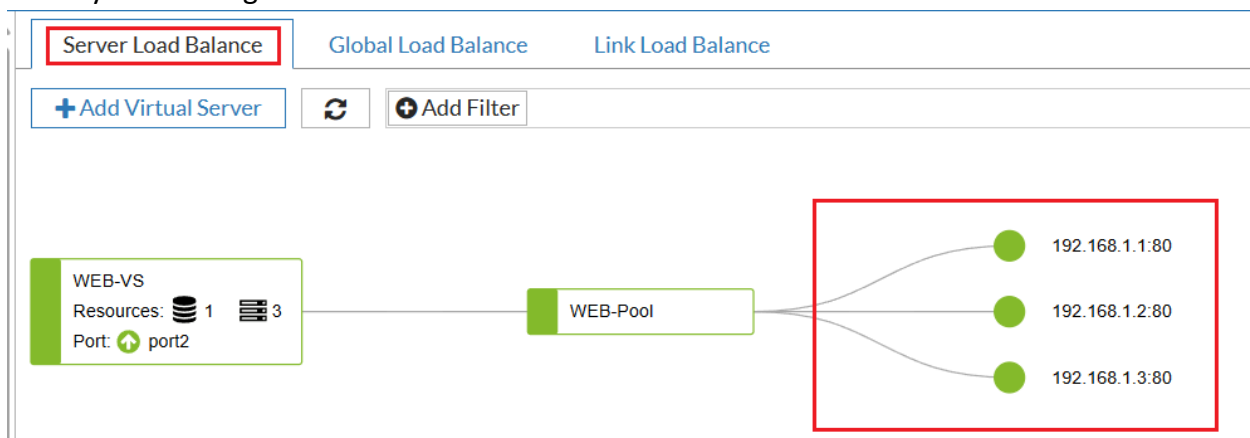


root@SRV1:~# service apache2 start

In this case TCP-based health check were used, FortiADC not able to detect the failure on Servers. While the **web service resources** on servers are not available, the server is still capable of responding to TCP packets. Consequently, FortiADC would interpret the server as healthy, as it relies on the TCP replies for status verification, missing the underlying application-level issue where the resources are not available.

```
root@SRV1:~# cd /var/www/html/  
root@SRV1:/var/www/html# ls  
index.html  
root@SRV1:/var/www/html# rm index.html
```

Navigate to **FortiView>Logical Topology>Server Load Balance** all the servers are up green and healthy even though the resources are not available



Configure Back the Server 1

```
#!/bin/bash  
sudo cat > /var/www/html/index.html << EOF  
<html>  
<head>  
<title>This is Server 1</title>  
</head>  
<body style="background-color:yellow;">  
<h1 style="color:red;">THIS IS SERVER 1</h1>  
</body>  
</html>  
EOF
```